

Predictive and Uncertainty-Aware Scheduling of Apache Iceberg Maintenance Under Concurrent Analytical WorkloadsUncertainty-Aware
Iceberg Maintenance Scheduling

Predictive and Uncertainty-Aware Scheduling of Apache Iceberg Maintenance Under Concurrent Analytical Workloads

Shashank Keshava Murthy
PES University
Bangalore, India

Prachi Jalan
PES University
Bangalore, India

Shashank D
PES University
Bangalore, India

Samarth
PES University
Bangalore, India

ABSTRACT

Modern analytical data platforms increasingly rely on open table formats such as Apache Iceberg to manage large-scale data lakes. To preserve read efficiency, these systems perform background physical-layout maintenance, including small-file compaction through `rewrite_data_files`. However, compaction competes with foreground analytical queries for CPU, executor memory, I/O bandwidth, and the operating-system page cache. Existing operational approaches commonly use fixed schedules, coarse resource thresholds, or layout-only optimization and therefore do not explicitly model the timing-dependent interference imposed on active workloads.

This paper presents an empirical and predictive framework for learned, uncertainty-aware timing of Apache Iceberg maintenance under concurrent analytical query execution. We conduct a four-phase study using Apache Spark and TPC-H workloads [? ?]. Phase 3A measures interference directly; Phase 3B evaluates pre-decision machine-learning regressors and a binary SLA classifier; Phase 3C evaluates uncertainty-aware scheduling policies; and Phase 3D evaluates leave-one-configuration-out cross-validation, trivial baselines, calibration, and zero-shot out-of-domain (OOD) tests.

Across Phase 3A, 528 query executions over 20 paired trials show mean Query Interference Ratios (QIRs) of 10.38% under FIFO and 12.77% under FAIR scheduling, with statistically significant paired degradation in both modes. The largest reported query-level degradation is 21.35% for Q14 under FAIR. In Phase 3B, a Random Forest regressor obtains a mean absolute error (MAE) of 5.38% for continuous QIR prediction. Binary SLA classification is less reliable: the draft reports a nominal ROC-AUC of 0.531 in one aggregate summary, while its detailed diagnostic table reports ROC-AUC 0.7205; this discrepancy is retained as an issue requiring resolution before submission. In Phase 3C, a conformal upper-bound policy [? ? ?] reduces the reported SLA violation rate from 13.7% for Always-Run to 0.6%, at the cost of allowing maintenance in only 18.5% of decision windows. Phase 3D shows substantial degradation of point predictors under unseen fragmentation states, while the reported 95% quantile model attains 95.0% empirical coverage on 80 OOD observations. These results motivate uncertainty-aware maintenance timing while also highlighting the need for larger, independently validated experiments before making distribution-free or general-purpose guarantees.

Shashank Keshava Murthy, Shashank D, Prachi Jalan, and Samarth.
Predictive and Uncertainty-Aware Scheduling of Apache Iceberg Maintenance Under Concurrent Analytical Workloads. PVLDB, 20(1): XXX-XXX, 2027.
doi:XX.XX/XXX.XX

1 INTRODUCTION

Modern analytical data architectures have evolved from traditional monolithic data warehouses toward decoupled “data lakehouses” built on object stores and open storage formats such as Apache Iceberg [?], Apache Hudi, and Delta Lake. These formats provide transactional semantics, schema evolution, and partition evolution over large datasets. Continuous streaming ingestion, micro-batch writes, and update/delete workloads can nevertheless degrade the physical layout of a table. In particular, repeated writes can create large numbers of small files, increasing metadata overhead and the cost of opening and scanning data files.

To preserve table health and query responsiveness, lakehouse engines execute background maintenance procedures. In Apache Iceberg [?], `rewrite_data_files` merges fragmented data files into larger target files. Although compaction can improve long-term layout quality, executing it concurrently with user queries introduces short-term resource contention: compaction reads many input files, decodes columnar data, creates new files, and consumes compute, memory, storage I/O, and cache resources.

1.1 The Operational Paradox

This creates a timing problem: maintenance is necessary to prevent long-term layout degradation, but its immediate execution may increase foreground query latency. The draft identifies four limitations in existing operational strategies:

- (1) **Static cron/off-peak rules.** Fixed schedules such as a daily maintenance window cannot reliably identify low-interference periods in continuously active, multi-tenant environments.
- (2) **Coarse resource thresholds.** Rules based only on CPU or memory utilization do not encode query structure, storage fragmentation, or the characteristics of the active execution stage.

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 20, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

- (3) **Layout-only optimization.** Prior compaction and layout systems can optimize what to compact and how to organize files, but the draft argues that they do not explicitly model the interference penalty imposed on concurrent foreground queries.
- (4) **Uncertainty-blind learning.** A point predictor can become overconfident when the workload or table structure differs from the training distribution. This motivates explicit uncertainty estimation and bounded deferral.

1.2 Research Question

We study the following question:

Given that lakehouse table maintenance must be executed to preserve long-term read health, when should it be scheduled relative to active query workload dynamics to reduce foreground interference while retaining useful maintenance throughput?

1.3 Contributions

The study makes four main contributions.

- **Empirical interference quantification.** A counterbalanced experimental harness measures concurrent compaction and TPC-H query execution, with temporal-overlap validation.
- **Leakage-isolated prediction.** The feature matrix separates signals available before the scheduling decision from post-execution telemetry, allowing continuous QIR prediction without using future information.
- **Uncertainty-aware scheduling.** Several policies combine point or quantile prediction with a bounded starvation mechanism and split-conformal calibration.
- **Generalization analysis.** LOCO-CV and zero-shot OOD tests expose the limits of point prediction under structural distribution shift and evaluate empirical coverage of an upper-quantile predictor.

2 BACKGROUND AND RELATED WORK

2.1 Apache Iceberg Layout Mechanics and Compaction

An Apache Iceberg table snapshot consists of metadata, manifest lists, manifest files, and columnar data files, commonly Parquet or ORC. Repeated ingestion can create a fragmented layout. The draft summarizes the small-file problem as

$$\text{File Count} \gg \frac{\text{Total Table Size}}{\text{Target File Size}}. \quad (1)$$

Apache Iceberg provides `rewrite_data_files` for compaction. The draft describes BinPack as approximately solving

$$\min K \quad \text{s.t.} \quad \sum_{i \in \text{Bin}_k} S_i \leq S_{\text{target}}, \quad k = 1, \dots, K, \quad (2)$$

where S_i is an input-file size and S_{target} is the target output size. The maintenance operation can nevertheless be expensive because it reads many input files and rewrites their contents.

2.2 Apache Spark Scheduling Mechanics

Apache Spark exposes FIFO and FAIR task scheduling [?]. FIFO gives earlier jobs priority for available task slots, whereas FAIR scheduling allocates tasks across named pools according to configured shares. A simplified allocation ratio is

$$\text{Slot Allocation Ratio} = \frac{w_f}{w_f + w_b}, \quad (3)$$

where w_f and w_b are the foreground and background pool weights.

Task-pool isolation does not necessarily imply hardware-level isolation: foreground and background tasks may still share CPU caches, memory bandwidth, storage queues, and the operating-system page cache.

2.3 Positioning

The intended distinction is that the present study treats maintenance timing as a prediction problem whose target is concurrent-query interference, rather than treating maintenance only as a layout-selection problem.

3 SYSTEM ARCHITECTURE AND PROBLEM FORMULATION

3.1 Query Interference

Let $\mathcal{W} = \{q_1, \dots, q_M\}$ be an analytical workload. Let $T_{\text{baseline},i}$ be the latency of query q_i without concurrent maintenance and $T_{\text{concurrent},i}$ its latency when maintenance operation $\mathcal{M}$ executes concurrently. We define Query Interference Ratio as

$$\text{QIR}_i = \frac{T_{\text{concurrent},i} - T_{\text{baseline},i}}{T_{\text{baseline},i}} \times 100\%. \quad (4)$$

For a decision window W , the draft defines Workload Interference Ratio as

$$\text{WIR} = \frac{\sum_{i \in W} T_{\text{concurrent},i} - \sum_{i \in W} T_{\text{baseline},i}}{\sum_{i \in W} T_{\text{baseline},i}} \times 100\%. \quad (5)$$

For a user-defined SLA threshold τ_{SLA} ,

$$y_{\text{SLA},i} = \mathbb{I}(\text{QIR}_i > \tau_{\text{SLA}}). \quad (6)$$

Unless otherwise stated, the experiments use $\tau_{\text{SLA}} = 15\%$.

3.2 Decision Space

At each decision time t , the scheduler selects

$$a_t \in \{\text{NOW}, \text{DEFER}, \text{FORCED_OVERRIDE}\}. \quad (7)$$

- **NOW:** execute maintenance immediately when predicted interference is within the permitted SLA threshold.
- **DEFER:** postpone maintenance to a later decision window when predicted interference is too high.
- **FORCED_OVERRIDE:** execute maintenance after the deferral streak reaches a configured starvation bound.

Table 1: Positioning of the studied approach relative to the systems named in the draft [? ? ? ?].

Framework	Focus	Dynamic workload model	Interference model	Conformal uncertainty
AutoComp	Compaction selection/thresholding	No	No	No
Smart Compaction	Cost/benefit optimization	Partial	No	No
PTO	Partition/layout optimization	No	No	No
Karim et al.	RL for maintenance	Yes	Partial	No
This work	Predictive maintenance timing	Yes	Yes	Yes

3.3 Causal Feature Isolation

A central design constraint is causal availability. Measurements generated during or after maintenance cannot be used to make the decision that launched that maintenance. The draft partitions features into

$$X_{\text{total}} = X_{\text{pred}} \cup X_{\text{eval}}, \quad X_{\text{pred}} \cap X_{\text{eval}} = \emptyset. \quad (8)$$

The pre-decision matrix contains table fragmentation and size statistics, 1-Hz host telemetry, storage I/O indicators, baseline duration, workload type, and scheduler mode. The evaluation-only matrix contains quantities such as compaction duration, JVM GC time, executor deserialization time, and post-run compaction I/O.

The scheduler therefore computes

$$\widehat{\text{QIR}}_t = f_{\theta}(X_{\text{pred},t}). \quad (9)$$

4 PHASE 3A: EMPIRICAL MEASUREMENT OF WORKLOAD INTERFERENCE

4.1 Experimental Setup

Experiments were conducted on a dedicated Ubuntu Linux testbed using Java 11 OpenJDK, Apache Spark 3.3.4 in standalone mode, and Apache Iceberg 1.4.3 over local Hadoop storage. The benchmark uses TPC-H SF1, with 6,001,215 records in lineitem. The draft reports a 3.25% coefficient of variation over 20 isolated baseline repetitions.

4.2 Experimental Harness

The measurement pipeline resets the table to a known layout, creates fragmented treatment layouts, launches `rewrite_data_files`, verifies overlap using the service interface, executes the foreground workload, and computes paired QIR values. Temporal overlap is explicitly measured as

$$\text{OverlapRatio}_i = \frac{\text{Duration}(Q_i \cap M)}{\text{Duration}(Q_i)}. \quad (10)$$

A concurrent run is retained when $\text{OverlapRatio}_i \geq 0.95$. The draft reports that all retained FIFO and FAIR concurrent executions satisfied this criterion.

4.3 Aggregate Results

The draft reports 528 query executions and 44 physical compaction runs.

The largest query-level degradation reported is Q14: 20.95% under FIFO and 21.35% under FAIR. Q6, Q18, Q3, Q1, and Q12 exhibit smaller but positive degradation in at least one scheduler mode.

A paired comparison between FIFO and FAIR QIR is reported as $p = 0.16689$ with $d_z = -0.3214$. The draft therefore interprets FAIR task pools as not providing a statistically significant reduction in interference.

5 PHASE 3B: PREDICTIVE MODELING AND NEGATIVE RESULTS

5.1 Parameter Sweep

Phase 3B spans three fragmentation levels (50, 200, and 500 files), two workload intensities (single-stream Q14 and a six-query TPC-H batch), and two scheduler modes (FIFO and FAIR). The sweep contains 168 paired observations and 3,670 task-level telemetry records sampled at 1 Hz.

5.2 Continuous QIR Prediction

Models are evaluated using grouped cross-validation by configuration ID.

The Random Forest [?] is the strongest continuous predictor in the reported Phase 3B results.

5.3 Binary SLA Classification

The draft also evaluates a Random Forest classifier for $\text{QIR} > 15\%$. The text reports nominal accuracy near 78–81%, while the detailed diagnostic table reports the following metrics.

The draft attributes the weak classification behavior to class imbalance (23 positive observations out of 168), boundary noise around the 15% SLA threshold, and the loss of information caused by binarizing a continuous interference quantity.

Important data-consistency issue. The abstract and contribution list state $\text{ROC-AUC} = 0.531$, whereas Table 5 states $\text{ROC-AUC} = 0.7205$ for the original and threshold-tuned models. These cannot both be the reported result. The final paper should replace the conflicting value after checking the experiment logs.

6 UNCERTAINTY-AWARE SCHEDULING POLICIES

6.1 Policy Suite

Seven policies are evaluated:

- (1) Always-Run.
- (2) Always-Defer.
- (3) Random ($P = 0.5$).

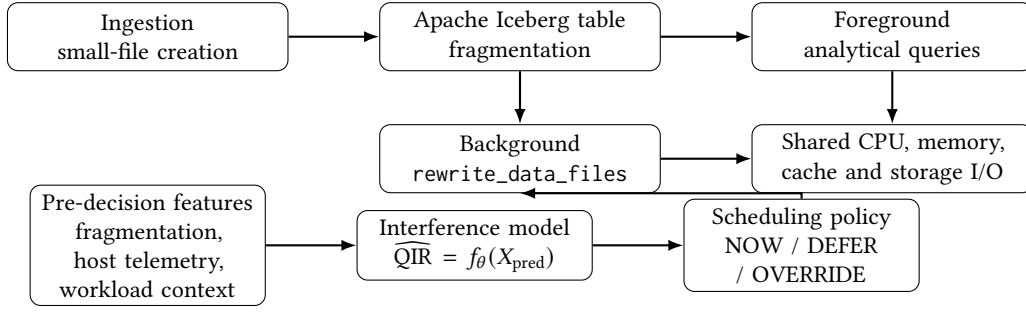


Figure 1: High-level architecture of the studied maintenance-timing framework. The model uses only information available before the scheduling decision.

Table 2: Phase 3A aggregate interference results reported in the draft.

Scheduler	Baseline (ms)	Concurrent (ms)	Mean QIR	95% CI	Wilcoxon p	d_z
FIFO	17,416	19,120	10.38%	[6.94, 13.82]	0.00161	1.1651
FAIR	18,984	21,394	12.77%	[10.54, 14.99]	< 0.00001	2.4966

Table 3: Selected query-level latency degradation.

Query	Baseline	FIFO	FIFO QIR	FAIR QIR
Q14	1,858	2,239	20.95%	21.35%
Q6	1,727	2,041	19.11%	17.95%
Q18	4,681	5,132	10.03%	13.76%
Q3	2,279	2,444	10.16%	10.31%
Q1	4,564	4,947	8.90%	10.86%
Q12	2,307	2,317	0.78%	9.26%

Table 4: Continuous QIR prediction results.

Model	MAE	RMSE
Ridge	6.91%	8.86%
Lasso	7.35%	9.19%
Quantile ($q = 0.95$)	10.81%	12.16%
Random Forest	5.38%	7.34%

Table 5: Binary SLA-classification diagnostics reported in the draft.

Variant	Acc.	Bal. Acc.	Prec.	Rec.	AUC
Original RF	80.95	54.21	.2353	.1739	.7205
Tuned threshold	77.98	56.15	.2308	.2609	.7205
Weighted RF	73.21	62.53	.2500	.4783	.6843

- (4) Explicit resource heuristic: $\text{CPU} < 40\%$ and $\text{DiskIOPS} < 500$.
- (5) Predictive mean-QIR: $\widehat{\text{QIR}}_{RF} \leq 15\%$.
- (6) Conservative quantile: $\widehat{\text{QIR}}_{0.95} \leq 15\%$.
- (7) Split-conformal upper bound: $\widehat{\text{QIR}}_{\text{conf}} \leq 15\%$.

6.2 Bounded Starvation

All predictive policies maintain a deferral counter K_{defer} and force execution when it reaches $K_{\text{max}} = 3$:

$$a_t = \begin{cases} \text{FORCED_OVERRIDE}, & K_{\text{defer}} \geq K_{\text{max}}, \\ \text{PolicyAction}(X_{\text{pred},t}), & \text{otherwise.} \end{cases} \quad (11)$$

This constraint prevents a scheduler from indefinitely deferring maintenance under persistent foreground pressure.

6.3 Policy Results

The policy results reveal a clear throughput–risk tradeoff: allowing maintenance more often exposes the workload to more interference, while conservative policies reduce reported SLA violations at the cost of more deferral. The draft describes the conservative quantile policy as Pareto optimal; however, a formal Pareto-front calculation is not supplied, so this claim should be verified or softened in the final manuscript.

7 PHASE 3D: FALSIFICATION, CALIBRATION, AND OOD GENERALIZATION

7.1 LOCO Cross-Validation

LOCO-CV holds out one configuration at a time from the 12-configuration sweep. The reported mean MAEs are 6.55% for Random Forest, 7.54% for Ridge, 7.66% for Lasso, and 10.81% for the $q = 0.95$ quantile regressor.

7.2 Trivial Baselines

The draft compares the learned models against a global mean QIR of 3.64%, workload-specific means of 12.50% for single-stream and 2.16% for batch workloads, and a training median of 6.83%.

Thus, in the reported LOCO experiment, the Random Forest is the only learned model that improves upon the strongest scalar baseline.

Table 6: Policy results across the 168-window decision trace.

Policy	Allowed	Deferred	Mean QIR	P95 QIR	SLA violations	Max streak
Always-Run	100.0	0.0	3.64	15.50	13.7	0
Always-Defer	0.0	100.0	0.00	0.00	0.0	168
Random	57.1	42.9	1.91	12.59	7.7	5
Resource heuristic	66.7	33.3	2.05	11.67	6.0	12
Predictive mean-QIR	91.7	8.3	3.05	15.04	10.7	12
Conservative quantile	31.0	69.0	0.52	5.75	1.2	27
Conformal upper bound	18.5	81.5	0.33	3.67	0.6	47

Table 7: LOCO-CV MAE across the 12 configurations.

Held-out configuration	RF	Ridge	Lasso	Quantile $q = .95$
frag50_single_FIFO	12.16	13.82	13.82	16.59
frag50_single_FAIR	10.99	10.96	11.23	14.18
frag50_multi_FIFO	5.37	5.86	6.13	7.91
frag50_multi_FAIR	4.88	5.62	5.81	7.53
frag200_single_FIFO	10.45	11.77	11.77	14.07
frag200_single_FAIR	8.89	9.72	9.94	12.03
frag200_multi_FIFO	4.12	4.95	5.08	6.84
frag200_multi_FAIR	3.95	4.61	4.75	6.42
frag500_single_FIFO	7.15	8.43	8.56	10.82
frag500_single_FAIR	5.82	6.91	7.02	9.15
frag500_multi_FIFO	2.58	3.98	3.89	7.21
frag500_multi_FAIR	2.22	3.91	3.88	7.02
Overall mean	6.55	7.54	7.66	10.81

Table 8: LOCO performance against scalar baselines.

Model/baseline	MAE	RMSE
Random Forest	6.55	7.96
Training median	6.83	8.44
Global mean	7.12	8.79
Ridge	7.54	8.94
Lasso	7.66	9.01
Quantile $q = .95$	10.81	12.16

Table 9: Zero-shot OOD results reported in the draft.

Model	MAE	RMSE	R^2
Ridge	22.46	31.95	-0.49
Random Forest	25.40	36.77	-0.98
Lasso	28.19	36.65	-0.96
Quantile $q = .95$	29.76	39.77	-1.31

7.3 Split-Conformal Calibration

Let $\mathcal{D}_{\text{cal}}$ be a calibration set. The draft defines the nonconformity residual for an upper quantile predictor $[?]$ as

$$E_i = \text{QIR}_i - \widehat{\text{QIR}}_{0.95,i}. \quad (12)$$

The conformal upper bound is

$$\widehat{\text{QIR}}_{\text{conf}} = \widehat{\text{QIR}}_{0.95} + \widehat{q}_{\text{conf}}, \quad (13)$$

where $\widehat{q}_{\text{conf}}$ is computed from the empirical calibration residual quantile.

The reported LOCO results state that the raw $q = 0.95$ predictor attains 90.00% empirical coverage and that conformal adjustment raises this to 93.75%, with an offset of +1.38 QIR percentage points.

7.4 Zero-Shot OOD Evaluation

The OOD experiment introduces unseen fragmentation states of 100 and 350 files. Eighty new physical trials are evaluated without retraining.

The quantile model is reported to cover 76 of 80 observations, corresponding to 95.00% empirical coverage. The point predictors have negative R^2 under the same structural shift.

Interpretation caveat. Exact empirical 95.00% coverage on this 80-observation OOD sample is an observed property of this test set; by itself, it is not a proof of distribution-free coverage under arbitrary future distribution shifts. In particular, the manuscript should distinguish finite-sample empirical coverage from a formal distribution-free guarantee, and it should clearly state whether the evaluated bound is the raw quantile model or the conformally calibrated bound.

8 LIMITATIONS AND THREATS TO VALIDITY

- (1) **Scale.** The experiments use TPC-H SF1 with 6,001,215 line item records. Larger distributed clusters may exhibit different bottlenecks.
- (2) **Engine scope.** The study uses Apache Spark 3.3.4. Trino, DuckDB, ClickHouse, and other execution engines may expose different interference patterns.
- (3) **Workload generation.** The draft uses controlled Poisson arrivals. Production workloads can be highly non-stationary and may require temporal sequence models.

- (4) **Small OOD sample.** The OOD evaluation contains only 80 trials, so the reported 95.00% empirical coverage should not be treated as a universal guarantee.
- (5) **Internal result inconsistencies.** The draft contains conflicting AUC values and mixes claims about conformal coverage with results attributed to the raw quantile model. These must be resolved from experiment logs.
- (6) **Missing artifact details.** The current draft does not provide enough information about code, exact hardware specifications, random seeds, train/calibration/test partitioning, or full feature definitions for an independent reproduction.

9 CONCLUSION AND FUTURE WORK

This paper studies the timing of background Apache Iceberg maintenance under concurrent analytical workloads. The reported Phase 3A measurements show substantial query degradation when compaction overlaps foreground execution, with mean QIR values of 10.38% for FIFO and 12.77% for FAIR. The results also indicate that task-pool scheduling alone does not eliminate contention from shared physical resources.

For prediction, the Random Forest provides the lowest reported continuous-QIR MAE in Phase 3B and LOCO-CV. The policy experiments show a measurable tradeoff between maintenance throughput and SLA violation risk: conservative upper-bound policies allow fewer maintenance operations but substantially reduce the observed violation rate. The OOD experiment further demonstrates that point predictors can fail under unseen fragmentation states.

The strongest conclusion supported by the current evidence is therefore not that conformal prediction universally guarantees safe maintenance scheduling, but that uncertainty-aware upper-bound policies are a promising mechanism for controlling the risk of maintenance under workload and layout uncertainty. Future work should evaluate the scheduler on larger distributed clusters, richer workload traces, multiple engines, longer time horizons, and independently reproducible artifact packages.

REFERENCES

- [1] Apache Iceberg Project. 2026. Spark Procedures and rewrite_data_files Documentation. <https://iceberg.apache.org/docs/latest/spark-procedures/>.
- [2] Apache Spark Project. 2026. Job Scheduling and FAIR Scheduler Documentation. <https://spark.apache.org/docs/latest/job-scheduling.html>.
- [3] AutoComp Authors. 2024. AutoComp: Automatic Compaction for Distributed Log-Structured Lakehouses. In *Proceedings of the VLDB Endowment*. Bibliographic details should be replaced with the verified publication record before submission.
- [4] Leo Breiman. 2001. Random Forests. *Machine Learning* 45 (2001), 5–32. doi:10.1023/A:1010933404324
- [5] A. Karim et al. 2026. Reinforcement Learning for Autonomous Storage Maintenance in Cloud Databases. Preprint; bibliographic details should be verified before submission.
- [6] Roger Koenker and Gilbert Bassett. 1978. Regression Quantiles. *Econometrica* 46, 1 (1978), 33–50.
- [7] PTO Authors. 2025. Partitioning and Layout Optimization for Open Table Formats. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*. Bibliographic details should be replaced with the verified publication record before submission.
- [8] Glenn Shafer and Vladimir Vovk. 2008. A Tutorial on Conformal Prediction. *Journal of Machine Learning Research* 9 (2008), 371–421.
- [9] Smart Compaction Authors. 2025. Cost-Aware Compaction Scheduling in Cloud Lakehouses. In *Proceedings of the IEEE International Conference on Data Engineering*. Bibliographic details should be replaced with the verified publication record before submission.
- [10] Transaction Processing Performance Council. 2026. TPC-H Benchmark Specification, Version 3.0.1. <https://www.tpc.org/tpch/>.
- [11] Vladimir Vovk and Claus Bendtsen. 2018. Conformal Predictive Decision Making. In *Proceedings of Machine Learning Research*, Vol. 91. 52–62.
- [12] Vladimir Vovk, Jing Shen, Valery Manokhin, and Min-Ge Xie. 2017. Nonparametric Predictive Distributions Based on Conformal Prediction. In *Proceedings of Machine Learning Research*, Vol. 60. 82–102.